ЛАБОРАТОРНАЯ РАБОТА №1

Как это устроено, простыми словами

Короткое введение — семь шагов от нуля

Здесь только суть: что просят сделать, почему это работает и что отвечать на защите. Подробный разбор с формулами, дизассемблером и ссылками на источники — в отдельном документе.

1 Что вообще просят сделать

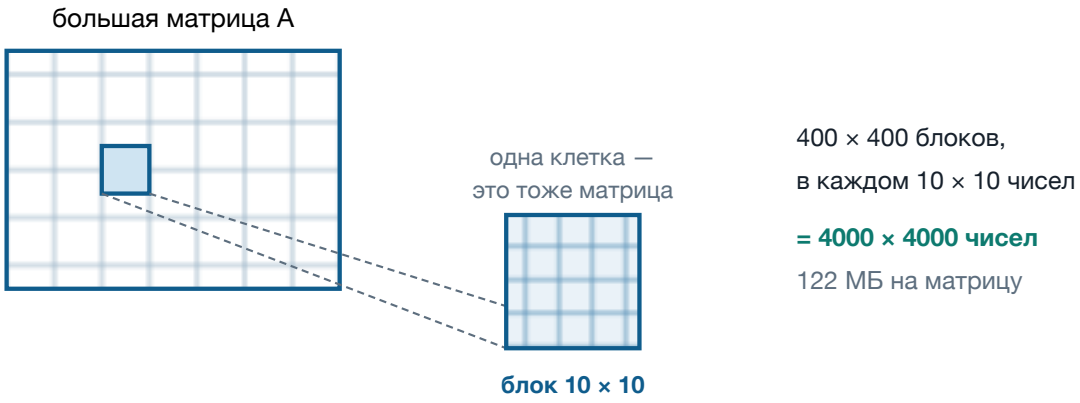
Нужно перемножить две матрицы. Но не одним способом, а **тремя**, и сравнить их между собой:

ВЕРСИЯ	ЧТО ЭТО
Скалярная	обычный код, векторизация выключена — «медленный эталон»
C1 — автоматическая	тот же обычный код, но компилятор сам ускоряет его как умеет
C2 — ручная	мы сами пишем ускоренную версию на специальных командах процессора

И доказать две вещи: что **C1 и C2 дают абсолютно одинаковый результат**, и что **наша ручная версия не медленнее автоматической**.

Матрицы не простые, а блочные

Обычная матрица — таблица чисел. Здесь каждая клетка большой таблицы — это сама маленькая таблица 10 × 10. Как коробка с конфетами: большая коробка 400 × 400 ячеек, и в каждой ячейке — коробочка поменьше на 100 конфет.



ЗАЧЕМ ТАКАЯ СТРАННАЯ ПОСТАНОВКА

Блочность — не каприз преподавателя. Как мы увидим на шаге 4, работать с матрицей по маленьким кусочкам **в разы быстрее**, чем целиком. Условие задачи просто заставляет сразу написать правильно.

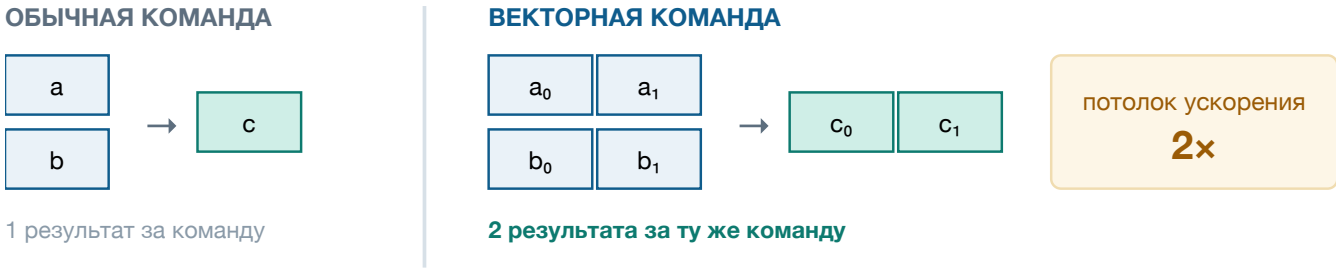
Чего делать нельзя

- **Транспонировать** матрицы (переворачивать строки в столбцы) — а это самый очевидный приём, и без него придётся думать.
- Использовать `clock()` и `time()` для замера — они слишком грубые.
- Печатать куски матриц на экран.

2 Что такое векторизация

Обычная команда процессора складывает **два числа** и получает одно. Но внутри процессора есть широкие регистры — «ячейки» по 128 бит. В такую ячейку помещается не одно число, а **два** числа типа `double`. И есть команды, которые складывают сразу обе пары за один раз.

> *Это как поливать грядку: можно из одной лейки по одному ростку, а можно взять трубу с двумя носиками и поливать два ростка одновременно. Усилие то же, скорость вдвое выше.*



Набор таких команд называется **SSE2**. Отсюда сразу понятно главное число работы: `double` занимает 64 бита, в регистр на 128 бит их влезает два, значит **быстрее чем в 2 раза стать невозможно**. (Для `float` влезло бы четыре, и потолок был бы 4×.)

Кто это делает — мы или компилятор

Компилятор умеет сам находить подходящие циклы и переписывать их на векторные команды. Это и есть версия **C1**. А в версии **C2** мы пишем векторные команды руками. Задание требует обе, чтобы сравнить: справился ли компилятор так же хорошо, как человек.

НАША ОСОБЕННОСТЬ

SSE2 — это команды процессоров Intel и AMD. Наш компьютер на Apple M5, у него другая архитектура, и команд SSE2 в нём **физически нет**. Мы обошли это библиотекой `sse2neon`: пишем код настоящими командами SSE2, а она при сборке заменяет каждую на точный аналог из набора NEON, который у M5 есть. Получается честный SSE2-код, который работает.

3 Почему процессор всё время ждёт

Кажется, что программа тормозит из-за вычислений. На самом деле процессор считает очень быстро, а вот **достать числа из памяти — медленно**. Настолько медленно, что почти всё время он просто ждёт.

Чтобы не ждать, рядом с процессором ставят маленькую быструю память — **кэш**.



Чем быстрее память — тем её меньше. Задача программиста: почаще попадать в верхние строчки.

> *Регистры — это то, что у вас в руках. Кэш — стол. Оперативная память — склад в соседнем здании. Сходить на склад в сто раз дольше, чем взять со стола.*

Память выдаётся не по одному числу

Важная деталь: из памяти нельзя забрать одно число. Она отдаёт данные **кусками по 128 байт** — это называется кэш-линия. В такой кусок помещается 16 чисел `double`.

Значит, есть два способа работать:

ЕСЛИ ВЗЯЛИ КУСОК И ИСПОЛЬЗОВАЛИ ВСЕ 16 ЧИСЕЛ	ЕСЛИ ИЗ КУСКА НУЖНО БЫЛО ОДНО ЧИСЛО
один поход «на склад» на 16 полезных чисел — отлично	один поход на 1 полезное число, остальные 15 выброшены — **в 16 раз хуже**

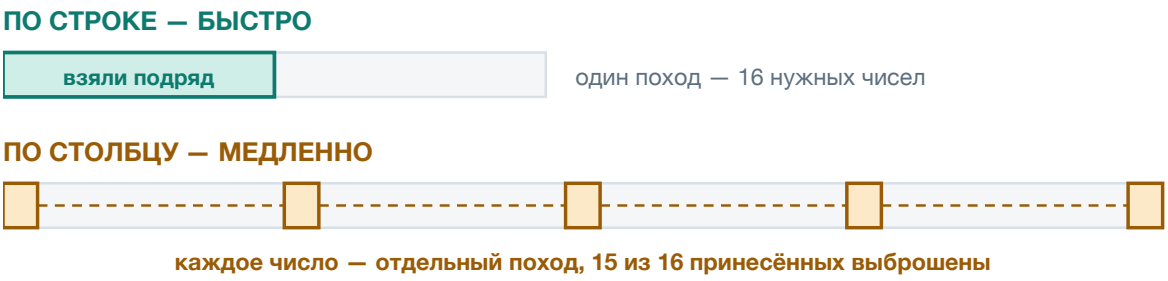
ОТСЮДА ВСЁ И СЛЕДУЕТ

Быстрая программа — это та, которая читает память **подряд** и использует принесённое **целиком**. Не «считает меньше», а «меньше бегает на склад».

4 Почему блоки — это быстро

Посмотрим, как умножают матрицы «в лоб». Чтобы получить одно число результата, надо взять **строку** первой матрицы и **столбец** второй.

И вот здесь ловушка. В памяти матрица лежит строками: сначала вся первая строка, за ней вся вторая. Поэтому идти по строке — значит читать подряд, это быстро. А идти по столбцу — значит прыгать через всю строку, каждый раз в новое место.



Очевидное решение — перевернуть вторую матрицу, чтобы столбцы стали строками. Но по условию задачи это запрещено.

Что делаем вместо этого

Раз переворачивать нельзя — уменьшим задачу. Вместо того чтобы возить числа со «склада» туда-сюда миллионы раз, мы берём **один маленький блок 10 × 10 и обсчитываем его целиком**, пока он лежит в кэше.

Блок 10 × 10 из `double` — это всего 800 байт. Кэш L1 — 64 000 байт. Три блока (два исходных и результат) занимают 2400 байт: помещаются с запасом в двадцать семь раз. Пока мы считаем блок, **на «склад» ходить вообще не нужно**.

И ещё одна хитрость: мы храним блоки в памяти **подряд, каждый целиком**. Тогда блок — это просто 800 байт, идущих друг за другом, и все принесённые куски памяти используются полностью.

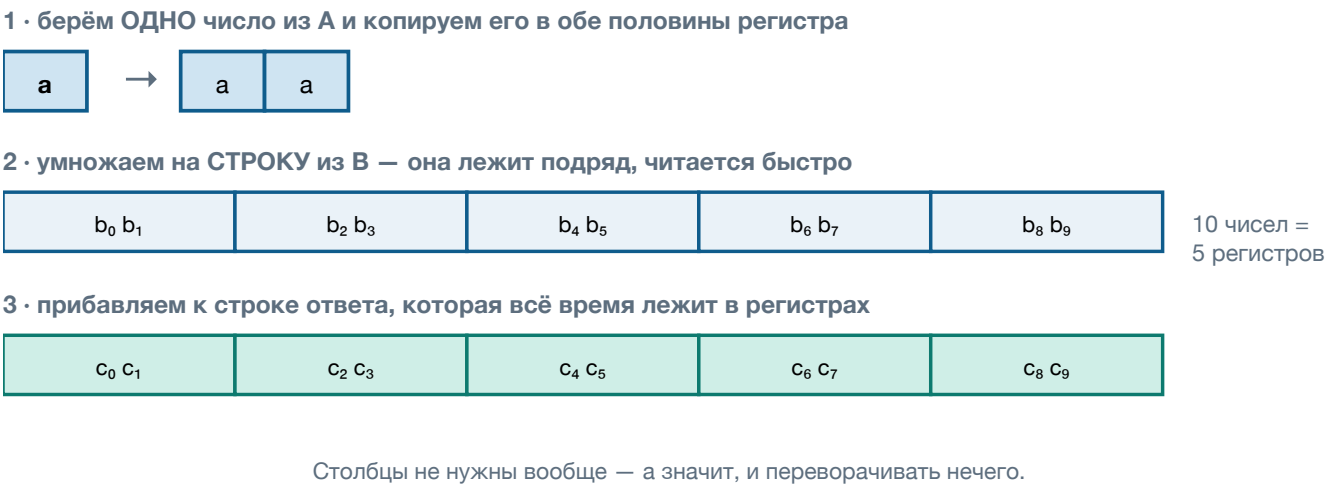
НАСКОЛЬКО ЭТО ПОМОГАЕТ

Очень сильно. Даже **без всякой векторизации** блочная версия работает в разы быстрее наивной, потому что процессор перестаёт ждать память. Векторизация добавляет свои 2× уже сверху.

5 Как умножать без переворота матрицы

Внутри блока проблема остаётся та же: формула требует строку × столбец. Решение придумали в Intel ещё в 1999 году, и оно неожиданно простое — **надо перевернуть не матрицу, а сам порядок вычислений**.

Вместо «возьмём строку и столбец, получим одно число» делаем так: **берём одно число из первой матрицы и умножаем его на целую строку второй**, а результат добавляем в строку ответа.



Столбцы не нужны вообще — а значит, и переворачивать нечего.

Математически результат ровно тот же — просто слагаемые собираются в другом порядке следования по памяти. Зато все чтения идут подряд, и запрет на транспонирование перестаёт мешать: он нам больше не нужен.

Так выглядит внутренний цикл ручной версии:

```
// a — одно число из A, размноженное на обе половины регистра
// дальше пять раз: взять два числа из строки B, умножить, прибавить
acc0 = _mm_add_pd(acc0, _mm_mul_pd(a, _mm_loadu_pd(row_b + 0)));
acc1 = _mm_add_pd(acc1, _mm_mul_pd(a, _mm_loadu_pd(row_b + 2)));
acc2 = _mm_add_pd(acc2, _mm_mul_pd(a, _mm_loadu_pd(row_b + 4)));
acc3 = _mm_add_pd(acc3, _mm_mul_pd(a, _mm_loadu_pd(row_b + 6)));
acc4 = _mm_add_pd(acc4, _mm_mul_pd(a, _mm_loadu_pd(row_b + 8)));
```

Пять «накопителей» `acc0…acc4` держат всю строку ответа прямо в регистрах процессора. Их пять, а не один, ещё и потому, что сложение занимает несколько тактов: пока считается первое, процессор успевает начать остальные четыре, и конвейер не простаивает.

6 Почему результаты совпадают до бита

Задание требует, чтобы C1 и C2 совпали **полностью**. Не «примерно», не «с точностью до сотых» — а байт в байт. И это не придирка, потому что с дробными числами так бывает не всегда.

> *Компьютер хранит дробные числа приближённо — как если бы вы записывали все суммы, округляя до копейки. Поэтому 0,1 + 0,2 у него не равно ровно 0,3. И если сложить те же слагаемые в другом порядке, ошибки округления лягут иначе и ответ отличится в последнем знаке.*

Отсюда правило: **переставлять порядок сложений нельзя**. Значит, ручная версия не имеет права «оптимизировать» порядок циклов — иначе она даст чуть-чуть другой результат и проверка провалится.

Три вещи, которые это гарантируют

ЧТО СДЕЛАНО	ЗАЧЕМ
Одинаковый порядок сложений	Обе версии складывают слагаемые в одной и той же последовательности. Каждая половинка векторного регистра повторяет ровно то, что делает обычный код.
Запрет «слитой» команды умножить-и-сложить	У процессора есть команда FMA, которая делает умножение и сложение за раз и округляет **один** раз вместо двух. Быстрее — но ответ отличается в последнем бите. Мы её запретили явным указанием компилятору.
Только целые числа от –8 до 8	Матрицы заполняются небольшими целыми. Сумма таких произведений остаётся целым числом, а целые числа компьютер хранит **точно**. Округления просто не возникает.

ЧТО ЭТО ДАЁТ

Первые два пункта делают совпадение гарантированным. Третий делает проверку **осмысленной**: раз округления нет вовсе, то любое расхождение — это настоящая ошибка в алгоритме, а не «не повезло с округлением». Поэтому сравнение делается самой строгой функцией — побайтовым `memcmp`.

7 Что получилось

Матрицы 4000 × 4000 чисел `double`, 122 МБ каждая, 128 миллиардов операций.

ВЕРСИЯ	ВРЕМЯ	УСКОРЕНИЕ
скалярная (векторизация выключена)	7,48 с	1,00×
C1 — автоматическая	3,84 с	1,95×
C2 — ручная, SSE2	**3,80 с**	**1,97×**

Напомним потолок из шага 2: для `double` быстрее 2× стать нельзя. Мы получили **1,97×** — то есть выжали 98 % возможного. Ручная версия ещё и чуть-чуть обогнала автоматическую, как и требовало задание.

Как доказать, что векторизация правда работает

Можно заглянуть в машинный код, который выдал компилятор, и просто посчитать команды:

ФАЙЛ	ВЕКТОРНЫХ КОМАНД	ОБЫЧНЫХ КОМАНД
автоматическая версия	100	0
ручная SSE2-версия	100	0
скалярная версия	0	200

Ровно вдвое меньше команд на ту же работу — потому что каждая обрабатывает два числа вместо одного. Лучшего доказательства не придумать.

НЕОЖИДАННАЯ НАХОДКА — ЕЁ СТОИТ РАССКАЗАТЬ НА ЗАЩИТЕ

В методичке отдельным пунктом стоит вопрос: «как отключить векторизацию???» Напрашивается флаг `-fno-vectorize`. Мы проверили — **он не делает ничего**, код остаётся векторным.

Причина в том, что векторизаторов у компилятора **два**. Первый работает с циклами. Но наш внутренний цикл короткий, всего 10 шагов, и компилятор разворачивает его в прямую цепочку команд ещё раньше — циклу просто неоткуда взяться. Зато вступает второй, **SLP**, который склеивает подряд идущие одиночные операции в векторные. Его гасит уже другой флаг — `-fno-slp-vectorize`.

Вывод: нужны **оба** флага. Если поставить только первый, «скалярный» эталон на самом деле останется векторным, и все замеры ускорения окажутся враньём.

Шпаргалка на защиту

Почему нельзя транспонировать и что вместо?

Это лишняя память и лишний проход копирования. Вместо — схема Intel: одно число из A умножаем на целую строку B. Столбцы не нужны.

Почему для double ускорение меньше, чем для float?

В регистр 128 бит влезает 2 числа double против 4 float. Потолок 2× против 4×.

Зачем блоки?

Блок 800 байт помещается в кэш L1 (64 КБ) с запасом. Пока считаем блок, в медленную память не ходим вообще.

Почему важен размер кэш-линии?

Память отдаёт данные кусками по 128 байт. Блоки лежат подряд, поэтому каждый кусок используется целиком, а не на 6 %.

Как доказали векторизацию?

Дизассемблер: 100 векторных команд против 200 обычных на тот же объём работы. Плюс отчёты компилятора.

Как отключили векторизацию?

Тот же файл собирается дважды, для эталона добавляются `-fno-vectorize` и `-fno-slp-vectorize`. Оба обязательны — первый сам по себе бесполезен.

Почему C1 и C2 совпали побитово?

Одинаковый порядок сложений, запрещён FMA, и данные — целые от −8 до 8, для которых округления не бывает.

Почему не используете FMA, он же быстрее?

Он округляет один раз вместо двух и даёт другой результат, а совпадение обязательно. И в самом SSE2 команды FMA нет.

Почему запрещён clock()?

Реальная точность около 10 мс, и на разных системах он означает разное. Используем `clock_gettime` и аппаратный счётчик.

Что такое rdtsc и где результат?

Счётчик тактов процессора, результат в паре регистров `edx:eax`. На нашем ARM его нет, вместо него системный счётчик `cntvct_el0`.

Где узкое место сейчас?

В вычислениях, не в памяти: блоки уже решили проблему памяти. Дальше мешают запрет FMA (осознанный) и длина цепочки сложений.